

FoodserviceDB Database Development and Query Analysis Report

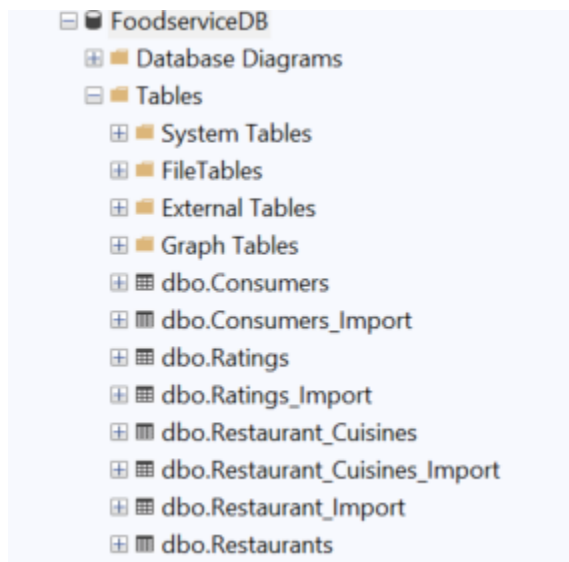
1. Database Creation and Data Import

1.1 Database Creation

A database named **FoodserviceDB** was created to store and manage information relating to restaurants, consumers, ratings, and restaurant cuisines. Four tables were created based on the supplied CSV files:

- Restaurants
- Consumers
- Ratings
- Restaurant_Cuisines

The original column names from the CSV files were retained to ensure compatibility between the imported data and the database structure. Primary keys and foreign keys were implemented to maintain data integrity and establish relationships between the tables.



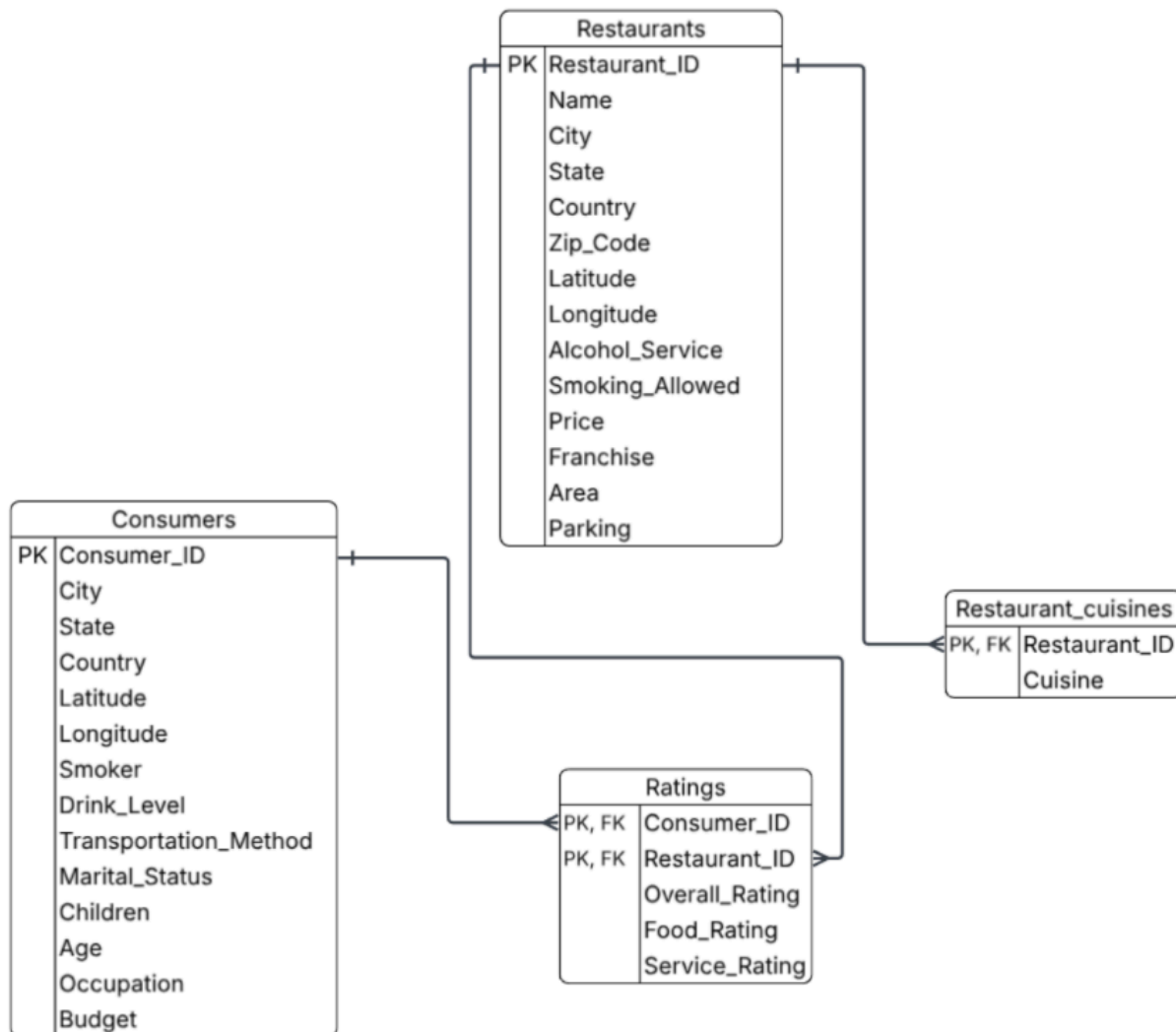
1.2 Database Relationships

The Restaurants table stores information relating to each restaurant, while the Consumers table stores information about customers. The Ratings table records ratings given by consumers to restaurants, and the Restaurant_Cuisines table stores the cuisines offered by each restaurant.

The following relationships were implemented:

- Restaurants and Ratings are linked through Restaurant_ID.
- Consumers and Ratings are linked through Consumer_ID.
- Restaurants and Restaurant_Cuisines are linked through Restaurant_ID.

Primary and foreign key constraints were used to ensure referential integrity and prevent invalid records from being inserted into the database.



1.3 Data Import

The data was imported from four CSV files into SQL Server. During the import process, the structure of each table was verified to ensure that the data types matched the imported values.

A review of the imported data identified NULL values within the Zip_Code column. As this field was not required for any of the analytical queries and did not affect referential integrity, the values were retained without modification.

```
✓ INSERT INTO Restaurants
  SELECT *
  FROM Restaurant_Import;

✓ INSERT INTO Consumers
  SELECT *
  FROM Consumers_Import;

✓ INSERT INTO Ratings
  SELECT *
  FROM Ratings_Import;

✓ INSERT INTO Restaurant_Cuisines
  SELECT *
  FROM Restaurant_Cuisines_Import;
```

2. Query Solutions

2.1 Restaurants with Medium Price, Open Area and Mexican Cuisine

This query was written to identify restaurants that satisfy three business requirements simultaneously: a medium price range, an open area environment, and Mexican cuisine. The Restaurants and Restaurant_Cuisines tables were joined using Restaurant_ID, and filters were applied to return only restaurants meeting all specified conditions.

```
87
88 SELECT
89     r.Restaurant_ID,
90     r.Name,
91     r.Price,
92     r.Area,
93     rc.Cuisine
94 FROM Restaurants r
95 JOIN Restaurant_Cuisines rc
96 ON r.Restaurant_ID = rc.Restaurant_ID
97 WHERE Price = 'Medium'
98 AND Area = 'Open'
99 AND rc.Cuisine = 'Mexican';
100
```

100 % 8 0 ↑ ↓ ◀

Results Messages

	Restaurant_ID	Name	Price	Area	Cuisine
1	135018	El Oceano Dorado	Medium	Open	Mexican
2	135106	El Rincón De Sa...	Medium	Open	Mexican

2.2 Comparison of Restaurants with an Overall Rating of 1

This query compares the number of Mexican and Italian restaurants that received an overall rating of 1. The Restaurants, Ratings, and Restaurant_Cuisines tables were joined using Restaurant_ID. Conditional aggregation and DISTINCT counting were used to ensure that each restaurant was counted only once, even if multiple customers gave the same rating.

The results showed that:

- Mexican restaurants with an overall rating of 1: **27**
- Italian restaurants with an overall rating of 1: **4**

The results indicate that considerably more Mexican restaurants received an overall rating of 1 than Italian restaurants within the dataset. This may suggest lower customer satisfaction among Mexican restaurants represented in the data, although additional analysis would be required to determine the underlying reasons.

```

105 SELECT
106     COUNT(DISTINCT
107         CASE
108             WHEN rc.Cuisine = 'Mexican'
109             THEN r.Restaurant_ID
110         END
111     ) AS Number_Of_Mexican_1Star_Restaurants,
112     COUNT(DISTINCT
113         CASE
114             WHEN rc.Cuisine = 'Italian'
115             THEN r.Restaurant_ID
116         END
117     ) AS Number_Of_Italian_1Star_Restaurants
118 FROM Restaurants r
119 JOIN Ratings ra
120     ON r.Restaurant_ID = ra.Restaurant_ID
121 JOIN Restaurant_Cuisines rc
122     ON r.Restaurant_ID = rc.Restaurant_ID
123 WHERE ra.Overall_Rating = 1;
124
125
126
127

```

00 % 8 0

Results Messages

	Number_Of_Mexican_1Star_Restaurants	Number_Of_Italian_1Star_Restaurants
1	27	4

2.3 Average Age of Consumers Giving a Service Rating of 0

This query calculated the average age of consumers who assigned a service rating of 0. The Consumers and Ratings tables were joined using Consumer_ID, and the AVG() function was used to calculate the average age.

The result returned an average age of **26 years**.

```

132
133 SELECT
134     ROUND(AVG(c.Age),0) Avg_Age
135 FROM
136     Consumers c
137 JOIN Ratings r
138     ON c.Consumer_ID = r.Consumer_ID
139 WHERE r.Service_Rating = 0;
140
141
142

```

100 %

✖ 8

⚠ 0

↑

↓

◀

Results

Messages

	Avg_Age
1	26

2.4 Restaurants Ranked by the Youngest Consumer

This query identified restaurants reviewed by the youngest consumer in the dataset. A nested query was used to determine the minimum consumer age, and the result set was sorted according to Food_Rating in descending order.

```

147 SELECT
148     DENSE_RANK() OVER(ORDER BY Food_Rating DESC) AS Restaurant_Rank,
149     r.Name,
150     ra.Food_Rating
151 FROM Restaurants r
152 JOIN Ratings ra
153     ON r.Restaurant_ID = ra.Restaurant_ID
154 JOIN Consumers c
155     ON ra.Consumer_ID = c.Consumer_ID
156 WHERE c.Age IN
157     (SELECT MIN(Age)
158      FROM Consumers)
159 ORDER BY ra.Food_Rating DESC;
160

```

100 %

✖ 5

⚠ 0

↑

↓

◀

Results

Messages

	Restaurant_Rank	Name	Food_Rating
1	1	Giovannis	2
2	1	Restaurant Bar Coty Y Pablo	2
3	2	El Cotorreo	1
4	2	Kiku Cuernavaca	1

2.5 Stored Procedure

A stored procedure named `Update_Service_Ratings_When_Parking_Available` was created to update the `Service_Rating` value to 2 for restaurants that have parking available. Restaurants with either 'Yes' or 'Public' listed in the `Parking` column were included in the update.

The procedure was successfully created and executed using the `EXEC` command. A verification query was then performed to confirm that the `Service_Rating` values had been updated correctly.

```
CREATE PROCEDURE Update_Service_Ratings_When_Parking_Available
AS
BEGIN
    UPDATE Ratings
    SET Service_Rating = 2
    WHERE Restaurant_ID IN
    (
        SELECT Restaurant_ID
        FROM Restaurants
        WHERE Parking IN ('Yes', 'Public')
    );
END;

EXEC Update_Service_Ratings_When_Parking_Available;
```

3. Additional Queries

3.1 Query Using the IN Operator

This query identified restaurants whose average food rating was above the overall average food rating across all restaurants. A nested query using the `IN` operator was used to compare restaurant performance against the overall dataset average.

The query helps identify restaurants that are performing better than average based on customer food ratings.

```
190
191 SELECT
192     Name
193 FROM Restaurants
194 WHERE Restaurant_ID IN
195 (
196     SELECT Restaurant_ID
197 FROM Ratings
198 GROUP BY Restaurant_ID
199 HAVING AVG(Food_Rating) >
200 (
201     SELECT AVG(Food_Rating)
202 FROM Ratings
203 )
204 );
205
```

100 % 5 0 ↑ ↓ ◀

Results Messages

	Name
1	Michiko Restaurant Japones
2	La Estrella De Dimas
3	Restaurant Las Mañanitas
4	Little Pizza Emilio Portes Gil
5	Giovannis

3.2 Query Using the EXISTS Operator

This query returned restaurants that have received at least one customer rating. The EXISTS operator was used to determine whether a related record exists in the Ratings table.

The results provide a list of restaurants that have received customer feedback and therefore have measurable performance data available.

208 SELECT
209 r.Restaurant_ID,
210 r.Name
211 FROM Restaurants r
212 WHERE EXISTS
213 (
214 SELECT 1
215 FROM Ratings ra
216 WHERE ra.Restaurant_ID = r.Restaurant_ID
217);

100 % 5 0 ↑ ↓ ◀

Results Messages

	Restaurant_ID	Name
1	132560	Puesto de Gorditas
2	132561	Cafe Ambar
3	132564	Church's
4	132572	Cafe Chaires
5	132583	McDonalds Centro
6	132584	Gorditas Doña Tota
7	132594	Tacos De Barbacoa Enfrente Del Tec
8	132608	Hamburguesas La Perica
9	132609	Pollo Frito Buenos Aires
10	132613	Carnitas Mata
11	132626	La Perica Hamburguesa
12	132630	Palomo Tec
13	132654	Carnitas Mata Calle 16 de Septiembre
14	132660	Carnitas Mata Calle Emilio Portes Gil
15	132663	Tacos Abi
16	132665	Tacos Correcaminos
17	132667	Little Pizza Emilio Portes Gil
18	132668	Tacos El Guero
19	132706	Gorditas Dona Tota
20	132715	Tacos De La Estacion
21	132717	Tortas Hawaii
22	132723	Gordas De Morales
23	132732	Taqueria El Amigo
24	132733	Little Caesars
25	132740	Carreton De Flautas Y Migadas
26	132754	Cabana Huasteca
27	132755	La Estrella De Dimas
28	132766	Mikasa

3.3 Query Using System Functions

This query calculated the average age of consumers within each budget category. The AVG() and ROUND() system functions were used to calculate and present the results.

The results provide insight into how customer age varies across different spending groups.

221	SELECT
222	Budget,
223	ROUND(AVG(Age),0) AS Average_Age
224	FROM Consumers
225	GROUP BY Budget
226	ORDER BY Average_Age DESC;

100 %	5	0	↑	↓	◀
-------	---	---	---	---	---

Results	Messages
---------	----------

	Budget	Average_Age
1	High	43
2	NULL	40
3	Low	26
4	Medium	25

3.4 Query Using GROUP BY, HAVING and ORDER BY

This query identified cuisine types offered by more than ten restaurants. The results were grouped by cuisine type, filtered using the HAVING clause, and sorted in descending order according to the number of restaurants offering each cuisine.

The query highlights the most common cuisine types available within the dataset.

229	SELECT
230	Cuisine,
231	COUNT(DISTINCT Restaurant_ID) AS Number_Of_Restaurants
232	FROM Restaurant_Cuisines
233	GROUP BY Cuisine
234	HAVING COUNT(DISTINCT Restaurant_ID) > 10
235	ORDER BY Number_Of_Restaurants DESC;

100 %	5	0	↑	↓	◀
-------	---	---	---	---	---

Results	Messages
---------	----------

	Cuisine	Number_Of_Restaurants
1	Mexican	28
2	Bar	13

4. Conclusion

The FoodserviceDB database was successfully created and populated using data imported from four CSV files. Primary and foreign key constraints were implemented to maintain referential integrity and accurately model relationships between restaurants, consumers, ratings, and cuisine types.

A range of SQL queries were developed to answer business questions relating to restaurant performance and customer behaviour. The project demonstrated the use of joins, nested queries, aggregation functions, system functions, sorting, grouping, filtering, and stored procedures. These techniques enabled meaningful insights to be extracted from the dataset and demonstrated the practical application of relational database concepts within SQL Server.